

PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Day 2 Hands-On Exercise

Your First OSDK Application

Phase 1 — Foundation Accelerator — Module 1

Author: Rajesh Pasham

1 September 2026

Table of Contents

Overview

This guide assumes no prior Palantir Foundry experience. Every screen, button, and menu you need to click is spelled out step by step. Where the exact wording of a button or menu might differ slightly depending on your Foundry version, a note tells you what to look for instead.

In this exercise you will create a Developer Console application, generate a TypeScript OSDK package for the Vehicle and Transponder object types you built on Day 1, set up a development environment (either inside Foundry or on your own machine), authenticate against your own enrollment, and read, filter, and write data entirely from code.

Estimated time: 60–75 minutes.

What You Will Build

- A Developer Console application scoped to your Vehicle and Transponder object types
- A generated TypeScript OSDK package, running either in a Foundry-hosted VS Code workspace or on your own machine
- A script that authenticates and lists your Active vehicles
- A new markVehicleForMaintenance action, created from scratch in Ontology Manager and called from your script
- (Stretch goal) A Python quick-start script performing the same read

Prerequisite: This exercise builds directly on the Ontology you created in Day 1's hands-on exercise. If you skipped a step there, revisit it before continuing — today's steps assume Vehicle and Transponder are already published and backed by data.

Prerequisites

- Completed Day 1 exercise: Vehicle and Transponder object types published and backed by data
- Your Foundry enrollment URL and login credentials
- A code editor if working locally — VS Code recommended (no editor needed if working in a Foundry workspace)
- (Local path only) Node.js 18 or later and npm installed on your machine
- (Stretch goal only) Python 3.10+ and pip installed locally

0. Foundry Navigation Basics

Before starting, here are the handful of navigation elements this entire guide refers to. Take a moment to find each one in your own browser window.

- **Foundry home page:** the page you land on right after logging in, at a URL like `https://your-enrollment.palantirfoundry.com`
- **App launcher:** a small grid icon (usually nine dots, sometimes a squares icon) in the top navigation bar. Clicking it opens a searchable list of every Foundry application you have access to, including Developer Console and Ontology Manager
- **Search resources bar:** a search box near the top of most Foundry applications, often with a keyboard shortcut of Ctrl+K (Windows/Linux) or Cmd+K (Mac). Use it to jump directly to an object type, application, or other resource by name
- **Left sidebar:** inside most Foundry applications (Ontology Manager, Developer Console), a vertical panel on the left lists the sections and resources of whatever you currently have open
- **Breadcrumb bar:** a horizontal trail near the top of the page (e.g. Discover > Vehicle > Actions) showing where you are and letting you click back to a previous level

If something looks different: If any icon or label described in this guide looks slightly different in your version of Foundry, use the search resources bar (Ctrl+K / Cmd+K) to jump straight to what you need by typing its name.

Exercise 1 — Open Developer Console

1. Log in to your Foundry enrollment and make sure you are on the Foundry home page.
2. Click the app launcher icon (the small grid of dots) in the top navigation bar.
3. In the search box that appears, type Developer Console.
4. Click the Developer Console result to open it. It opens either in the same tab or a new browser tab, depending on your settings.
5. Confirm you are in the right place: the page header should read Developer Console, and you should see either an empty state (“No applications yet”) or a list of applications if you or a colleague already created one.

If you don't see Developer Console: If Developer Console does not appear in your app launcher, it may not be enabled for your enrollment yet. Ask your Foundry administrator to enable the Developer Console application, or raise it in Office Hours.

Exercise 2 — Create a Developer Console Application

A Developer Console “application” is a container that bundles an OAuth client (for authentication) with a generated OSDK package (for code access to your Ontology). You will create one now.

2.1 Start the Application Wizard

1. In Developer Console, look for a button labeled + New application or Create application, usually near the top-right of the page.
2. Click it to open the application creation wizard.

2.2 Basic Information

1. On the first screen (often titled Basic information), enter a name for your application, e.g. Vehicle Ops Client.
2. Optionally choose an icon or color for the application — this is cosmetic and does not affect functionality.
3. Click Next (or Continue) to proceed.

2.3 Application Type

1. On the next screen (often titled Application type or Client type), choose Client-facing application. This is the option for applications where a person logs in through a browser — exactly what today's script will do.
2. Click Next.

2.4 Permissions & Redirect URL

1. On the Permissions screen, find the Grant type (or OAuth flow) setting and select Authorization code.
2. Find the field labeled Redirect URL(s) and enter `http://localhost:8080/auth/callback` exactly as written. This is the address your script will run on locally; Foundry redirects back to it after a successful login.
3. Click Next.

2.5 Review & Create

1. Review the summary screen showing the name, application type, and redirect URL you configured.
2. Click Create (or Finish) to create the application.
3. You are taken to the new application's detail page. Confirm the page header shows the application name you chose.

Save this value: On your application's detail page, look for a field labeled Client ID (a long string of letters and numbers) and note it down — you will need it in Exercise 5. It is usually shown on an Overview or Settings tab in the left sidebar of the application page.

Exercise 3 — Add Ontology Resources & Generate Your SDK

1. On your application's detail page, look in the left sidebar for a section called Ontology resources, Resources, or SDK (the exact label varies by Foundry version).
2. Click it to open the resource picker.
3. Click + Add resource (or Add object type).
4. In the search box, type Vehicle and select the Vehicle object type from the results.
5. Repeat: click + Add resource again, search for Transponder, and select it.
6. Look for a language selector on the same page (often a dropdown or set of tabs labeled TypeScript, Python, Java). Select TypeScript.
7. Click Save (or Generate SDK).
8. Wait for Foundry to generate the package — this usually takes under a minute. A progress indicator or notification will confirm when it is ready.

3.1 Find Your SDK Details

1. After generation completes, look for a tab labeled SDK or Install in the left sidebar of your application page.
2. On this tab, you should see: an install command (e.g. `npm install @your-enrollment/vehicle-ops-client-sdk`), your Foundry URL, and your Ontology RID (a long identifier starting with `ri.ontology...`).
3. Also on this page (or a nearby API Documentation tab), you can browse the exact generated methods and parameter names for Vehicle, Transponder, and any actions you add later — keep this tab open, you will refer back to it throughout today's exercise.

Save these four values: Write down (or copy into a scratch document) your Client ID, your Foundry URL, your Ontology RID, and the install command shown on this SDK tab. You will use all four in the next exercises.

Exercise 4 — Set Up Your Development Environment

Choose one of the two paths below. Path A runs entirely in your browser and needs no local installation. Path B runs on your own machine. Both lead to the same result — pick whichever you have available.

Path A — Foundry Code Repository & VS Code Workspace (Recommended)

This path uses Foundry's own hosted development environment, so there is nothing to install.

A.1 Create a Code Repository

1. Return to your application's detail page in Developer Console (use the app launcher or your browser's back button if you navigated away).
2. In the left sidebar, look for a section called Code Repository or Develop.
3. Click + Create code repository (or + New repository).
4. In the template picker, choose the React application template. This bootstraps a starter project already wired for authentication and your generated OSDK — you do not need to configure authentication by hand on this path.
5. Give the repository a name, e.g. vehicle-ops-client-app.
6. Click Create.

A.2 Open Your Workspace

1. Wait for Foundry to provision the repository — you will see a progress indicator; this typically takes one to two minutes.
2. Once ready, click the Open in VS Code button, usually near the top-right of the repository page.
3. A new browser tab opens with a full VS Code editor, running entirely in Foundry — no local installation required.

A.3 Orient Yourself in VS Code

1. On the far left, a narrow icon bar lets you switch between views (Explorer, Search, Source Control, and so on). The Explorer icon (usually the topmost, looking like stacked pages) is selected by default.
2. The Explorer panel lists your project's files and folders, including a src folder — this is where your application code lives.
3. Open the integrated terminal: click Terminal in the top menu bar, then New Terminal (or use the keyboard shortcut `Ctrl+`` on Windows/Linux, `Cmd+`` on Mac). A command-line panel opens at the bottom of the screen.
4. A live Preview pane, usually on the right side of the screen or in a separate browser tab, shows your running application and refreshes automatically as you edit code. Look for a Pop out or Open in new tab button if you want a larger view.

A.4 Locate Your Pre-Configured Client

1. In the Explorer panel, expand the src folder and look for a file with a name like `foundryClient.ts` or `client.ts` — it was generated automatically when your repository was created.
2. Open it and confirm it already contains a working `createClient(...)` call, with your Client ID, Foundry URL, and Ontology RID filled in for you.
3. Because authentication is already wired up on this path, skip Exercise 5's manual `auth.ts` setup steps — just note the name this file exports (commonly `client`) and continue to Exercise 5's `index.ts` code, importing `client` from this file's path instead of `./auth`.

Good to know: Everything you type into the integrated terminal (`npm install`, `npm start`, and so on) runs on Foundry's infrastructure, not your own machine — you can safely close your laptop lid overnight and pick up where you left off next time you open this workspace.

Path B — Local Machine

Use this path if you prefer working in your own editor or do not have access to Foundry Code Repositories.

1. Open a terminal application on your computer (Terminal on Mac, or Command Prompt / PowerShell / Windows Terminal on Windows).
2. Confirm Node.js is installed by typing `node --version` and pressing Enter. You should see a version number of 18 or higher. If you see an error instead, install Node.js from nodejs.org before continuing.
3. Create a new folder for this exercise, e.g. `mkdir vehicle-ops-client && cd vehicle-ops-client`.
4. Run `npm init -y` to create a package.json file.
5. Run `npm install typescript ts-node @types/node --save-dev` to add TypeScript tooling.
6. Run `npm install @osdk/client @osdk/oauth` to add the OSDK client and OAuth helper libraries.
7. Install your generated SDK package using the exact install command you saved from Exercise 3 (it will look similar to `npm install @your-enrollment/vehicle-ops-client-sdk`, but use the real command shown on your application's SDK tab).
8. Run `npx tsc --init` to create a tsconfig.json file.
9. Open the vehicle-ops-client folder in your code editor (in VS Code: File → Open Folder...).

package.json (reference)

```
{
  "name": "vehicle-ops-client",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "ts-node src/index.ts"
  },
  "dependencies": {
    "@osdk/client": "^2.0.0",
    "@osdk/oauth": "^1.0.0"
  },
  "devDependencies": {
    "typescript": "^5.4.0",
    "ts-node": "^10.9.0",
    "@types/node": "^20.11.0"
  }
}
```

Exercise 5 — Authenticate & Fetch Your Vehicles

If you are on Path A (Foundry workspace), your authentication is already configured — skip step 1 below and use the client export from the file you located in Exercise 4, Step A.4. If you are on Path B (local), complete every step.

- (Path B only) Create a file `src/auth.ts` and paste the authentication code below, replacing the three placeholders with the Client ID, Foundry URL, and Ontology RID you saved from Exercise 3.
- Create a file `src/index.ts` with the script that lists your vehicles, shown below.
- (Path A) In the integrated terminal, run `npm start`. (Path B) In your terminal, run `npm start`.
- A browser tab opens asking you to sign in with your Foundry credentials — do so.
- After signing in, check your terminal (Path B) or the Preview pane / terminal output (Path A) and confirm your 12 vehicles are printed.

`src/auth.ts` (Path B only)

```
import { createPublicOauthClient } from "@osdk/oauth";
import { createClient } from "@osdk/client";

const auth = createPublicOauthClient(
  "<YOUR_CLIENT_ID>",
  "<YOUR_FOUNDRY_URL>",
  "http://localhost:8080/auth/callback",
);

export const client = createClient(
  "<YOUR_FOUNDRY_URL>",
  "<YOUR_ONTOLOGY_RID>",
  auth,
);
```

`src/index.ts`

```
// Path B: import { client } from "./auth";
// Path A: import { client } from "./foundryClient"; (or the filename you found in
// Exercise 4)
// Replace "@your-enrollment/vehicle-ops-client-sdk" with your
// generated package name from Exercise 3.
import { Vehicle } from "@your-enrollment/vehicle-ops-client-sdk";
import { client } from "./auth";

async function listActiveVehicles() {
  const page = await client(Vehicle)
    .where({ status: { $eq: "Active" } })
    .fetchPage({ $orderBy: { year: "desc" } });

  console.log(`Found ${page.data.length} active vehicles:`);
  for (const vehicle of page.data) {
    console.log(
      `${vehicle.vehicleId} — ${vehicle.make} ${vehicle.model} (${vehicle.year}),`
      `${vehicle.currentLocation}`,
    );
  }
}

listActiveVehicles().catch(console.error);
```

Troubleshooting: If nothing prints, double-check the redirect URL configured in Exercise 2.4 matches `http://localhost:8080/auth/callback` exactly, including the port number.

Exercise 6 — Filter and Sort Transponders

Extend your script to answer a second question: which transponders are currently unassigned?

1. Add a second function to `src/index.ts`, `listAvailableTransponders`, following the pattern below.
2. Call it alongside `listActiveVehicles` at the bottom of the file.
3. Run `npm start` again and confirm the 1 unassigned transponder from Day 1's dataset (TX-40004) is printed.

```
import { Transponder } from "@your-enrollment/vehicle-ops-client-sdk";

async function listAvailableTransponders() {
  const page = await client(Transponder)
    .where({ status: { $eq: "Unassigned" } })
    .fetchPage({ $orderBy: { receivedDate: "asc" } });

  console.log(`Found ${page.data.length} unassigned transponders:`);
  for (const t of page.data) {
    console.log(` ${t.transponderId} — ${t.type}, received ${t.receivedDate}`);
  }
}
```

Exercise 7 — Define and Call Your First Action Type

So far you have only read data. Now you will define an Action Type entirely from scratch in Ontology Manager, add it to your SDK, and call it from code. Every click is spelled out below.

7.1 Navigate to Ontology Manager

1. Click the app launcher icon (grid of dots) in the top navigation bar.
2. Type Ontology Manager in the search box and click the result.
3. You land on the Discover page, showing a grid of recently viewed object types — you should recognize Vehicle and Transponder from Day 1.

7.2 Open the Vehicle Object Type

1. In the left sidebar, under the Resources heading, click Object types.
2. In the search box at the top of the object type list, type Vehicle.
3. Click on the Vehicle result to open its detail page.
4. You should now see a left-hand tab list for this specific object type: Overview, Properties, Security, Datasources, Links, Actions, and others.

7.3 Start a New Action Type

1. In the left-hand tab list (for the Vehicle object type, not the main Ontology Manager sidebar), click Actions.
2. Click the + New Action Type button, usually in the top-right corner of the Actions panel.
3. A configuration screen opens for your new action.

7.4 Name the Action

1. Find the field labeled API name and type `markVehicleForMaintenance` exactly (lowerCamelCase, no spaces).
2. Find the field labeled Display name and type a human-readable name, e.g. Mark Vehicle For Maintenance.

7.5 Add Parameters

1. Find the Parameters section of the configuration screen and click + Add parameter.
2. Set this parameter's API name to `vehicle`. For its Type, choose Object reference (sometimes labeled Object or Link), and when prompted for which object type, select Vehicle. This tells Foundry the action needs to know which vehicle to modify.
3. Click + Add parameter again. Set this second parameter's API name to `newStatus`, and set its Type to String. This will hold the new status value the caller wants to set.

7.6 Define the Effect (What the Action Actually Does)

1. Scroll to the section of the screen labeled Logic, Rules, or Effects (the exact label varies by version).
2. Click + Add rule (or + Add effect) and choose Modify object (sometimes labeled Edit object).
3. When asked which object to modify, select the vehicle parameter you created in step 7.5.
4. In the property mapping area, find the status property and set its new value to come from a parameter — choose `newStatus` from the dropdown of available parameters, rather than typing a fixed value.

7.7 Save and Publish

1. Click Save, usually in the top-right corner of the screen.
2. If a validation summary appears, review it for errors (a red icon indicates something needs fixing; a green checkmark means you're ready to proceed).
3. Click Publish to make the action available for use.

4. Confirm markVehicleForMaintenance now appears in the Vehicle object type's Actions list with a Published status badge.

If your screen looks different: Action Type configuration screens are one of the areas most likely to differ slightly between Foundry versions. If a label doesn't match exactly, look for the closest equivalent (e.g. “Effects” instead of “Logic”) — the underlying steps (name the action, add parameters, add a modify-object rule, save, publish) are consistent.

7.8 Add the Action to Your Developer Console Application

1. Return to Developer Console (app launcher → Developer Console) and open your Vehicle Ops Client application.
2. Navigate back to the same Ontology resources / SDK section you used in Exercise 3.
3. Click + Add resource, search for markVehicleForMaintenance, and select it.
4. Click Save (or Generate SDK) again — this bumps your SDK package to a new version that now includes the action.

7.9 Refresh Your Local Package

1. (Path A — Foundry workspace) In your VS Code integrated terminal, re-run the same install command from Exercise 3 to pull in the updated package.
2. (Path B — local machine) In your terminal, re-run the same install command from Exercise 3.
3. Open the API Documentation tab on your application's SDK page and note the exact generated method name and parameter names for markVehicleForMaintenance — use these to double-check the code below matches your version.

7.10 Call the Action From Code

Add this function to src/index.ts, call it at the bottom of the file, and run npm start once more.

```
import { markVehicleForMaintenance } from "@your-enrollment/vehicle-ops-client-sdk";

async function sendToMaintenance(vehicleId: string) {
  await client(markVehicleForMaintenance).applyAction({
    vehicle: vehicleId,
    newStatus: "Maintenance",
  });
  console.log(`${vehicleId} marked for maintenance.`);
}

// Try it on one of your Day 1 vehicles, e.g. "VEH-006"
sendToMaintenance("VEH-006").catch(console.error);
```

1. Run your script and confirm the confirmation message is printed.
2. Switch back to Ontology Manager, open the Vehicle object type, click Objects (or Explore), find VEH-006, and confirm its status property now reads Maintenance.

Troubleshooting: If applyAction is rejected, re-check the exact parameter names shown in your generated API Documentation from step 7.9 — they must match exactly, including capitalization.

Exercise 8 (Stretch Goal) — Python Quick-Start

If time allows, reproduce the read from Exercise 5 in Python using token-based authentication — useful for notebooks and scripts.

1. In Foundry, generate a personal access token from your user settings (or use the token your enrollment provides for local development).
2. In a new folder, run `pip install foundry-platform-sdk` (or the package name shown in your enrollment's Python OSDK documentation).
3. Create `vehicles.py` with the script below, substituting your enrollment hostname and token.
4. Run `python vehicles.py` and confirm the same active vehicles are printed.

```
from foundry_sdk import FoundryClient
from foundry_sdk.auth import UserTokenAuth

auth = UserTokenAuth(hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")
client = FoundryClient(auth=auth, hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")

active_vehicles = client.ontology.objects.Vehicle.where(
    client.ontology.objects.Vehicle.status == "Active"
)

for vehicle in active_vehicles.iterate():
    print(vehicle.vehicleId, vehicle.make, vehicle.model, vehicle.currentLocation)
```


Appendix A — Full src/index.ts Reference

The complete file combining Exercises 5–7:

```
import { Vehicle, Transponder, markVehicleForMaintenance } from
"@your-enrollment/vehicle-ops-client-sdk";
import { client } from "./auth";

async function listActiveVehicles() {
  const page = await client(Vehicle)
    .where({ status: { $eq: "Active" } })
    .fetchPage({ $orderBy: { year: "desc" } });

  console.log(`Found ${page.data.length} active vehicles:`);
  for (const vehicle of page.data) {
    console.log(
      `${vehicle.vehicleId} - ${vehicle.make} ${vehicle.model} (${vehicle.year}),
      ${vehicle.currentLocation}`
    );
  }
}

async function listAvailableTransponders() {
  const page = await client(Transponder)
    .where({ status: { $eq: "Unassigned" } })
    .fetchPage({ $orderBy: { receivedDate: "asc" } });

  console.log(`Found ${page.data.length} unassigned transponders:`);
  for (const t of page.data) {
    console.log(`${t.transponderId} - ${t.type}, received ${t.receivedDate}`);
  }
}

async function sendToMaintenance(vehicleId: string) {
  await client(markVehicleForMaintenance).applyAction({
    vehicle: vehicleId,
    newStatus: "Maintenance",
  });
  console.log(`${vehicleId} marked for maintenance.`);
}

async function main() {
  await listActiveVehicles();
  await listAvailableTransponders();
  await sendToMaintenance("VEH-006");
}

main().catch(console.error);
```

Appendix B — Quick Navigation Reference

A condensed lookup of every navigation path used in this guide, for quick reference during the exercise.

Task	Navigation Path
Open Developer Console	App launcher (grid icon) → search “Developer Console”
Open Ontology Manager	App launcher (grid icon) → search “Ontology Manager”
Create an application	Developer Console → + New application
Add object types to SDK	Application page → Ontology resources / SDK → + Add resource
Create a Code Repository	Application page → Code Repository → + Create code repository
Open VS Code workspace	Code Repository page → Open in VS Code
Find an object type	Ontology Manager → Object types → search by name
Create an Action Type	Object type page → Actions tab → + New Action Type
Verify an object's data	Ontology Manager → Object type → Objects / Explore tab

Definition of Done — Day 2

You have completed today's hands-on exercise when:

- A Developer Console application exists, scoped to Vehicle, Transponder, and markVehicleForMaintenance
- You have a working development environment — either a Foundry Code Repository / VS Code workspace, or a local project — that authenticates successfully
- Running your script prints all Active vehicles, sorted by year
- Running your script prints all unassigned transponders
- You created the markVehicleForMaintenance action type yourself, from scratch, in Ontology Manager
- Calling sendToMaintenance updates a real Vehicle's status, visible in Ontology Manager
- (Stretch) The Python script reproduces the vehicle read using token-based authentication

Looking ahead: Keep this project — Day 3 extends it into a React application using the same OSDK package and authentication pattern.